

Benchmarking the request time of Laravel, ASP.NET Core and Django

James Judd Follow 4 min read · Aug 4, 2018

421 2

One consideration when choosing a framework for web applications you build is how fast will requests be handled. To test this, I have created a typical API endpoint in a few popular web frameworks and then benchmarked the results.

The Application

The application will be an API which reads some rows from a MySQL database, and returns them in a JSON format – A typical CRUD endpoint.

I've made all of the applications available on my GitHub, please see the resources section.

Frameworks

I'll be using these three frameworks:

- [ASP.NET Web API](#) – This runs on Microsoft's open-source .NET core 2.1
- [Laravel 5.6](#) – The most popular PHP framework – PHP 7.2
- [Django 1.11](#) – A popular Python web framework – Python 3.6 (3.7 had an issue running this version of Django)

(UPDATE Nov 2018: [Part two of this article](#) include Adonis, a Node.js

(UPDATE Nov 2018: [Part two of this article](#) include Adonis, a Node.js framework)

Performance

My tutor at university always used to correct me when I used the word performant. As a standalone word, it is far too generic. Are we talking about memory usage, execution time, CPU cycles used, etc. For this comparison, I'll be measuring the duration of a request. The apache benchmarking tool (ab) is super handy for tasks like this. It will send concurrent requests and aggregate the results for you.

Docker

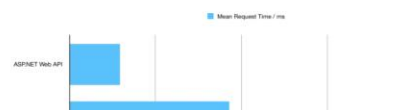
I've dockerised these applications so that they can be deployed with ease on any OS— I'd encourage any keen readers to clone these repos, run the containers and benchmark yourself too.

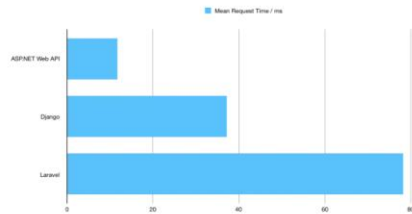
Development Web Servers

It is important to not use development web servers to test response times. Development web servers are often single-threaded and are not optimised for performance at all – so their response times would not be reflective of production. Let's be good scientists and make the test as close to production as possible.

Results

The full breakdown is below but here is a visual representation of the results (in this instance, smaller is better):





ASP.NET

```
ab -n 100 -c 10 http://localhost:8080/api/books
```

Output:

```
Concurrency Level:      10
Time taken for tests:    0.117 seconds
Complete requests:      100
Failed requests:         0
Total transferred:      29200 bytes
HTML transferred:       15300 bytes
Requests per second:    854.91 [#/sec] (mean)
Time per request:       11.697 [ms] (mean)
Time per request:       1.170 [ms] (mean, across all concurrent requests)
Transfer rate:          243.78 [Kbytes/sec] received

Connection Times (ms)
  min  mean(+-sd)  median    max
Connect:    0      0  0.1  0      1
Processing:  7     11  2.2  11     21
Waiting:    7     11  2.2  10     21
Total:      7     11  2.1  11     21
```

Laravel:

```
ab -n 100 -c 10 http://127.0.0.1/api/books
```

Document Path: /api/books

Document Length: 155 bytes

Concurrency Level: 10

Time taken for tests: 0.780 seconds

Complete requests: 100

Failed requests: 0

Total transferred: 40600 bytes

HTML transferred: 15500 bytes

Requests per second: 128.21 [#/sec] (mean)

Time per request: 77.995 [ms] (mean)

Time per request: 7.799 [ms] (mean, across all concurrent requests)

Transfer rate: 50.83 [Kbytes/sec] received

```
Connection Times (ms)
  min  mean(+-sd)  median    max
Connect:    0      0  0.1  0      1
Processing: 36     71 13.7  66    112
Waiting:   36     71 13.7  66    112
Total:     36     71 13.7  66    112
```

Django Results:

```
Server Software:      gunicorn/19.9.0
Server Hostname:      localhost
Server Port:          8000

Document Path:        /books
Document Length:      240 bytes

Concurrency Level:     10
Time taken for tests:  0.372 seconds
Complete requests:     100
Failed requests:        0
Total transferred:     42200 bytes
HTML transferred:      24000 bytes
Requests per second:   268.74 [#/sec] (mean)
Time per request:      37.211 [ms] (mean)
Time per request:      3.721 [ms] (mean, across all concurrent requests)
Transfer rate:         110.75 [Kbytes/sec] received

Connection Times (ms)
  min  mean(+-sd)  median    max
Connect:    0      0  0.1  0      1
Processing: 17     35  6.5  36     53
Waiting:   17     35  6.5  36     53
Total:     17     35  6.5  36     53
```

Django Results:

```
Server Software: gunicorn/19.9.0
Server Hostname: localhost
Server Port: 8000
Document Path: /books
Document Length: 240 bytes
Concurrency Level: 10
Time taken for tests: 0.372 seconds
Complete requests: 100
Failed requests: 0
Total transferred: 42200 bytes
HTML transferred: 24000 bytes
Requests per second: 268.74 [#/sec] (mean)
Time per request: 37.211 [ms] (mean)
Time per request: 3.721 [ms] (mean, across all concurrent requests)
Transfer rate: 110.75 [Kbytes/sec] received

Connection Times (ms)
  min  mean[+/-sd] median max
Connect: 0  0  0.1  0  1
Processing: 17  35  6.5  36  53
Waiting: 16  35  6.4  35  52
Total: 17  36  6.5  36  53
```

Conclusion

I was very impressed with how low the request time was for ASP.NET, but I did expect it to be superior, given that C# is a compiled language, and Python and PHP are interpreted languages.

I'd also like to say that you should not choose a framework based on these results, this is just a single factor amongst many; How easy is a framework to use? Does the framework have good documentation? Is there a wealth of third-party packages available? I may cover some of those topics at a later date.

Thanks for reading, please applaud and perhaps comment if you made it this far!

Conclusion

I was very impressed with how low the request time was for ASP.NET, but I did expect it to be superior, given that C# is a compiled language, and Python and PHP are interpreted languages.

I'd also like to say that you should not choose a framework based on these results, this is just a single factor amongst many; How easy is a framework to use? Does the framework have good documentation? Is there a wealth of third-party packages available? I may cover some of those topics at a later date.

Thanks for reading, please applaud and perhaps comment if you made it this far!

Resources:

- [.NET Core Application GitHub](#)
- [Django Application GitHub](#)
- [Dockerize a .NET Core application](#)
- [Quickstart: Compose and Django](#)

Web Development Laravel Python PHP AspNet

421 2

Written by James Judd
40 Followers · 23 Following
Backend Developer @ Canva. Working with CloudFlare, AWS, Terraform.
@juddling